



정보통신개론

“4주차 강의”

윤홍수

2025. 09. 23

Table of Contents

I. 2025년 2학기 4주차 강의 계획

- 프로그래밍 언어의 역사
- 언어 분류: 어셈블리어, 고급언어
- 퀴즈

프로그래밍 언어의 역사

■ 프로그래밍 언어란?

- ✓ 컴퓨터에게 특정 작업을 수행하도록 명령을 내리기 위해 인간이 사용하는 형식화된 언어
- ✓ 사람이 이해할 수 있는 자연어(Natural Language)와는 달리,
논리적이고 명확한 문법(Syntax)과 의미(Semantics)를 갖는 구조화된 언어
- ✓ 컴퓨터는 기본적으로 0과 1로 구성된 기계어(Machine Code)만 이해할 수 있기 때문에,
인간이 작성한 코드를 컴퓨터가 이해하도록 번역(Translation)하는 과정이 필요

프로그래밍 언어의 역사

■ 프로그래밍 언어란?

✓ 기능

- 명령어 전달 (Instruction Delivery)

- 사용자가 원하는 동작을 컴퓨터에게 정확하게 명령하는 도구

- 예: 변수 선언, 연산 수행, 조건 판단, 반복 실행 등

- 알고리즘 표현 (Algorithm Expression)

- 문제 해결 절차인 알고리즘을 정형화된 문법으로 표현

- 수학적 사고를 바탕으로 논리적 흐름을 코드로 구현 가능

(변수, 반복, 조건), (조건문, 반복문, 함수, 재귀), (표준화된 코드)

프로그래밍 언어의 역사

■ 프로그래밍 언어란?

✓ 기능

- 하드웨어 제어 (Hardware Control)
 - 운영체제나 임베디드 시스템에서 CPU, 메모리, I/O 장치 제어 등 저수준 접근이 가능
 - 대표적으로 C언어나 어셈블리 언어
- 소프트웨어 구조화 (Abstraction & Modularity)
 - 대규모 프로젝트에 필요한 추상화, 모듈화, 객체지향적 설계 지원
 - 유지보수, 재사용성, 협업에 유리한 구조 제공

프로그래밍 언어의 역사

■ 프로그래밍 언어란?

✓ 기능

- 플랫폼 독립성 제공 (Portability)

- 자바(Java)나 파이썬(Python) 등 일부 언어는 운영체제나 하드웨어에 독립적인 실행이 가능
- 예: "Write Once, Run Anywhere"
- JVM(Java Virtual Machine) : 자바 프로그램을 실행해주는 가상 소프트웨어

프로그래밍 언어의 역사 - 질문

■ 프로그래밍 언어란?

✓ 기능

- 플랫폼 독립성 제공 (Portability) – 왜 탄생하게 되었을까?
 - 1990년대 이전에는 운영체제마다 프로그래밍 방식이 모두 달랐다
 - Windows용 프로그램 ↔ mac용 프로그램 ↔ 유닉스용 프로그램 → 별도 개발 필요
- 인터넷 활성화 시대에 들어서
- 양한 디바이스(PC, 노트북, 스마트폰, 서버)에서 같은 서비스 제공이 필요
- 하나의 코드 → 어디서든 실행이 가능한 언어가 절실했다

프로그래밍 언어의 역사 - 질문

■ 프로그래밍 언어란?

✓ 기능

- 플랫폼 독립성 제공 (Portability) – 왜 탄생하게 되었을까?
 - 글로벌 배포 (다양한 OS, 디바이스 서비스 제공, 웹, 앱, 게임 등)
 - 유지보수 비용 절감 (OS 상관 없이, 효율성 증가)
 - 빠른 배포
 - 보안/업데이트 통일화

“세계 시장에서 경쟁하기 위한 ‘생존의 무기’”

프로그래밍 언어의 역사

■ 프로그래밍 언어란?

✓ 중요성

- 소프트웨어 산업의 근간

- 소프트웨어는 프로그래밍 언어를 통해 구현됨
- 프로그래밍 언어가 없으면 모든 디지털 서비스는 불가능

- 기술 발전의 촉진제

- 프로그래밍 언어는 하드웨어 성능 향상, AI, IoT, AR/VR 등 최신 기술과의 연계를 가능하게 함
- 언어의 발전은 기술의 표현력, 성능, 안정성까지 좌우함

프로그래밍 언어의 역사

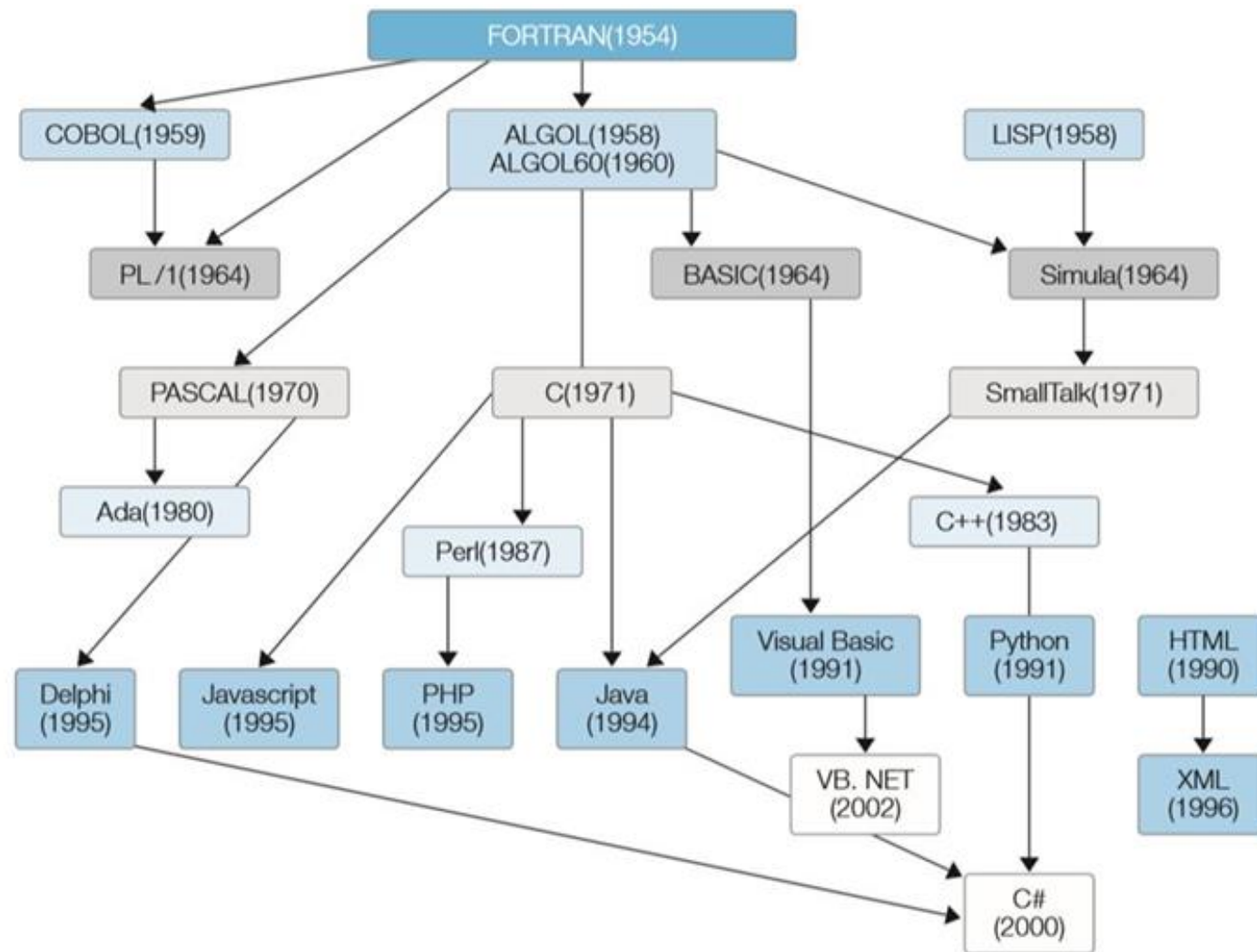
■ 프로그래밍 언어란?

✓ 중요성

- 컴퓨팅 사고력 증진

- 코딩을 통해 문제를 논리적으로 분해, 절차적으로 해결하는 훈련 가능
- 21세기 핵심 역량 중 하나로 평가됨 (Computational Thinking)

프로그래밍 언어의 종류



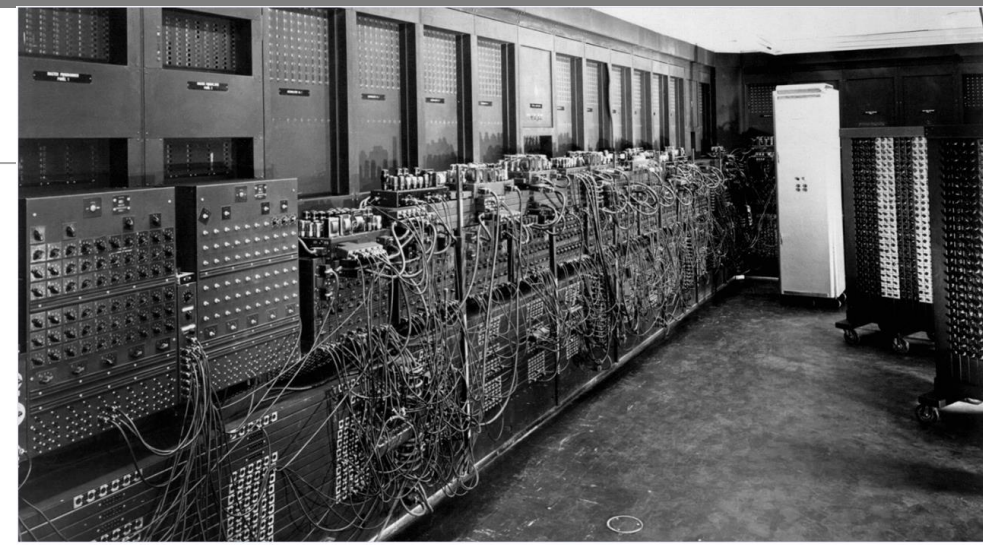
프로그래밍 언어의 역사

■ 컴퓨터 구조의 혁명

✓ 1940년대 이전 컴퓨터 구조

■ 하드와이어드 방식(Hardwired Architecture)

- 초기 컴퓨터는 특정 작업을 수행하도록 기계적으로 회로가 설계되어 있었음
- 예) 하나의 계산 작업을 위해 배선을 물리적으로 연결하거나 릴레이 회로를 변경
- 즉, 하나의 문제가 끝나면 하드웨어를 다시 재조립해야 했기 때문에
- 다른 작업을 하려면 새로운 회로 설계가 필요했고, 매우 비효율적이었음
- 대표 예시: ENIAC (1945)
 - 미국에서 개발된 세계 최초의 범용 전자식 디지털 컴퓨터
 - 덧셈, 곱셈, 루프 수행은 가능했지만
 - 새로운 프로그램을 실행하려면 전선 교체, 스위치 설정이 필요



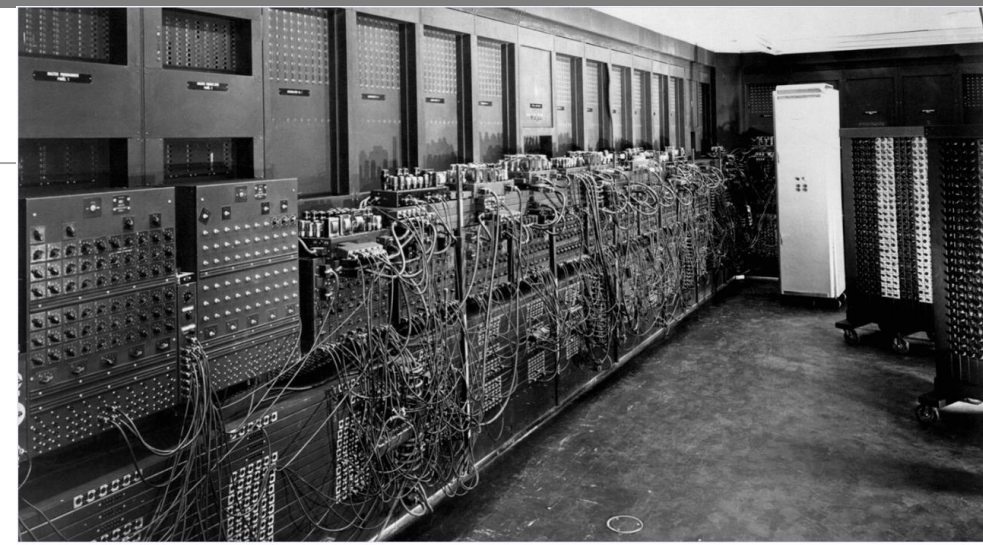
hardwired 방식을 사용했던 ENIAC. 출처: <https://www.sims lifecycle.com/2022/01/04/the-journey-of-eniac-the-worlds-first-computer/>

프로그래밍 언어의 역사

■ 컴퓨터 구조의 혁명

✓ ENIAC

- ENIAC은 2차 세계대전 중 포탄 탄도 계산을 자동화
- 당시 계산병들이 손으로 표를 그려야 했다
- 하루 종일 계산해도 오차가 많았고 너무 오래 걸렸다
- “인간이 너무 느리다. 계산을 자동으로 할 수는 없을까?”
 - 이 단순한 질문에서 전자식 계산기의 아이디어가 시작
 - 컴퓨터 과학의 시초이자 공학 + 논리 + 수학의 융합체



hardwired 방식을 사용했던 ENIAC. 출처: <https://www.sims lifecycle.com/2022/01/04/the-journey-of-eniac-the-worlds-first-computer/>

프로그래밍 언어의 역사

■ 컴퓨터 구조의 혁명

✓ 혁신적 의의

■ 프로그램의 유연성 획득

- 물리적 배선 변경 없이 소프트웨어 수정만으로 새로운 작업 수행 가능
- 사용자/개발자가 직접 코드를 작성하고 수정할 수 있게 됨

■ 프로그래밍 언어의 탄생 기반 마련

- 프로그램이 메모리에 저장되기 때문에, 프로그래밍 언어로 컴퓨터를 제어할 수 있는 기반 형성
- 이후 등장하는 어셈블리어, 고급 언어(FORTRAN, LISP 등)의 출현으로 이어짐

■ 현대 컴퓨터 구조의 표준

- 오늘날 우리가 사용하는 거의 모든 컴퓨터(PC, 스마트폰, 서버 등)는 폰 노이만 구조 또는 그 변형 구조를 기반으로 함

프로그래밍 언어의 역사

■ 플랑칼쿨 Plankalkül (1945)

- ✓ 개발자: 콘라트 츠제 (Konrad Zuse) – 독일 공학자
- ✓ Z3 는 콘라트 추제가 1938년에 설계 하고 1941년에 완성한 **독일의 전기기계식 컴퓨터**
- ✓ Z3 개발 이후, 복잡한 문제를 프로그램으로 표현할 수 있는 언어의 필요성을 느낌
- ✓ 당시에는 **FORTTRAN**도, **컴파일러**도, **프로그래밍 문법**도 존재하지 않았던 시절

Z3



뮌헨 독일 박물관 에 전시된 Zuse Z3 복제품

프로그래밍 언어의 역사

■ 플랑칼퀼 Plankalkül (1945)

- ✓ 개발자: 콘라트 츠제 (Konrad Zuse) – 독일 공학자
- ✓ 배경: 제2차 세계대전 중 독일에서 개발된 **세계 최초의 고급 프로그래밍 언어**
- ✓ 특징:
 - 변수, 타입
 - 배열, 구조체
 - 반복문, 논리 연산 등
 - 오늘날의 Python, C, Java의 기능이 이미 플랑칼퀼 초안에 담겨 있었던 셈

프로그래밍 언어의 역사

■ 플랑칼쿨 Plankalkül (1945)

✓ 특징:

- 컴퓨터가 본격 등장하기 전의 개념적인 설계
- 사용되지 못하고 1970년대에 재조명됨
- 의의 : 이후 등장한 프로그래밍 언어의 사상적 기반 제공

프로그래밍 언어의 역사

■ 플랑칼쿨 Plankalkül (1945)

✓ 기본 문법

$V0 + V1 \rightarrow R$

$V0, V1$: 입력 변수
 $\rightarrow R$: 결과를 변수 R 에 저장

$| V0 < V1 \rightarrow R = 1$
 $| V0 \geq V1 \rightarrow R = 0$

$V0$ 이 $V1$ 보다 작으면 R 에 1 저장, 아니면 0 저장

FOR $i = 0$ TO 9
 $V0[i] + 1 \rightarrow V0[i]$

배열 $V0$ 의 모든 요소에 1씩 더함

프로그래밍 언어의 역사

- 플랑칼퀼 Plankalkül (1945)

- ✓ 그 당시 왜 사용되지 못하고, 부각을 받지 못 했나?

- 컴퓨터의 발전 속도 보다, 개념적 이론이 앞섰다.(실행할 컴퓨터가 없었음)

- 컴파일러, 인터프리터가 존재하지 않음

- 제 2차 세계대전의 여파로, 독일 과학 고립

- 향후 재조명 ➔ 1970년대 이후에 재조명된 역사적 언어

프로그래밍 언어의 역사

■ 어셈블리어 (1949 ~)

✓ 기계어(Machine Code)를 사람이 이해하기 쉬운 형태로 바꾼 저수준 언어(Low-level Language)

✓ 특징:

- 기계어와 1:1 대응 : 명령 하나가 기계어 명령 하나와 정확히 대응
- 빠름 : 고급언어보다 빠른 실행 속도 가능
- 제어력 강함 : 하드웨어를 직접 제어 가능 (레지스터, I/O 포트 등)

✓ 의의:

- 운영체제, 드라이버, 임베디드 시스템에서 여전히 사용됨

```
MOV AX, 1  
ADD AX, 2
```

프로그래밍 언어의 역사

■ 어셈블리어 (1949 ~)

- ✓ 어셈블리어는 특정한 하나의 언어가 아니라, 각 CPU 아키텍처(명령어 집합 ISA: Instruction Set Architecture)에 따라 달라지는 저수준 언어(low-level language)
- ✓ 예
 - x86 어셈블리어: 인텔/AMD 계열 CPU용
 - ARM 어셈블리어: 스마트폰, 태블릿용 ARM 계열 CPU용
 - Z80 어셈블리어: 오래된 8비트 CPU용 (초기 닌텐도, 계산기, 1980년대 컴퓨터)

프로그래밍 언어의 역사

■ 어셈블리어 (1949 ~)

➤ 기본 문법

mov eax, 10 // 값 복사

add eax, ebx // 더하기

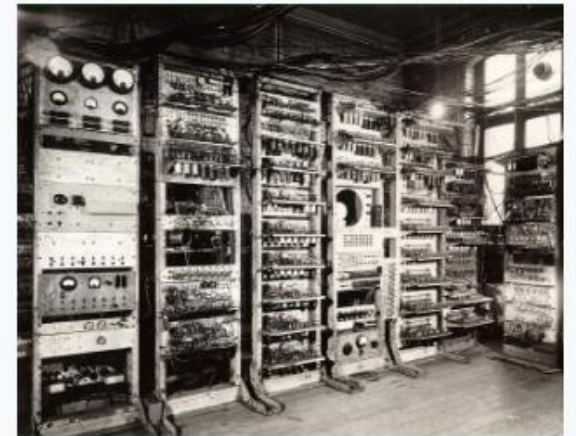
sub eax, 5 // 빼기

mov eax, 5 ; eax 레지스터에 5 저장
add eax, 4 ; eax에 4를 더함 ($5 + 4 = 9$)

프로그래밍 언어의 역사

- Autocode (1952) - 언어(BASIC, ALGOL, FORTRAN 등)에 영향
 - ✓ 1950년대 초, 영국 맨체스터 대학교에서 개발된 **세계 최초 수준의 고급 프로그래밍 언어 중 하나**
 - ✓ 컴퓨터: 맨체스터 마크 I(Manchester Mark I), Ferranti Mark I
 - ✓ 주요 개발자: Alick Glennie (앨릭 글레니)
 - ✓ 배경
 - 컴퓨터가 막 탄생한 초기 시기
 - 대부분의 프로그래밍은 기계어 또는 어셈블리 수준의 코드로 작성
 - 오류 발생률 높고, 코드 유지보수 어려움

Manchester Mark 1



The Manchester Mark 1 was one of the world's first stored-program computers

Product family	Manchester computers
Predecessor	Manchester Baby
Successor	Ferranti Mark 1

프로그래밍 언어의 역사

■ Autocode (1952)

✓ 목적

- 좀 더 읽기 쉽고, 명확한 표현을 통해 수학적 계산을 쉽게 프로그램으로 표현하기 위한 시도

✓ 특징

- 사용자가 작성한 코드를 기계어로 변환하는 컴파일러가 동작함 (최초의 컴파일러 언어 중 하나)
- 변수, 수식, 할당 등 수학 연산이 문법에 포함됨
- 사용자는 하드웨어 명령어를 몰라도 코딩 가능
- 이 문법 구조는 이후 BASIC, FORTRAN, ALGO 등에 영향을 줌

```
LET X = Y + Z  
PRINT X
```


프로그래밍 언어의 역사

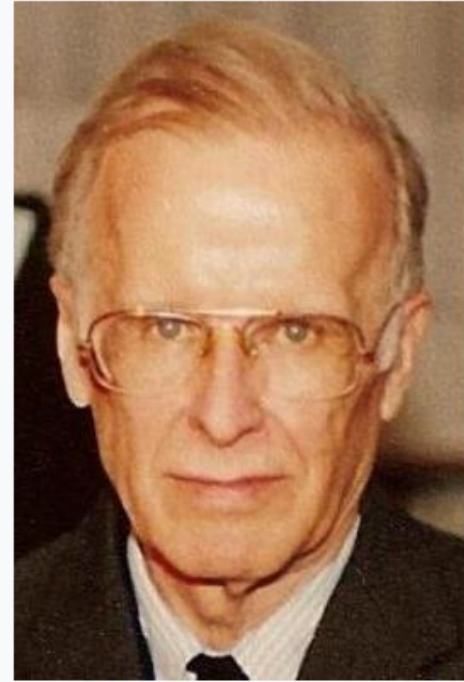
■ FORTRAN (1954) - 프로그래밍 언어 역사상 최초의 상용 고급 언어

✓ 개발자: IBM, 존 배커스

✓ 개발 배경

- 1950년대 당시, 과학자와 엔지니어들은 기계어 또는 어셈블리어로 수학 연산 코드를 작성
- 수식 하나 구현하려면 수십 줄 이상의 기계어 필요 → 매우 비효율적
- IBM은 과학자들을 위해, 수학 수식 그대로 컴퓨터가 이해할 수 있도록 도와주는 언어 개발 착수
- 목표: “사람이 수학적으로 표현한 공식을 컴퓨터가 이해하고 실행할 수 있게 하자”

존 배커스
John Backus



출생	1924년 12월 3일 필라델피아
사망	2007년 3월 17일(82세) 애슬랜드
출신 학교	컬럼비아 대학교 (M.S., 1950)
주요 업적	스피드코딩 포트란 알골 바쿠스-나우르 표기법 함수 수준 언어
수상	미국 과학 훈장 (1975) 튜링상 (1977)

프로그래밍 언어의 역사

■ FORTRAN (1954)

✓ 특징

- 컴파일러 기반
- 과학/공학 계산에 특화
- 배열, 반복문, 조건문, 함수 지원
- 플랫폼 독립성 확보 (여러 하드웨어 플랫폼에 이식 가능)

지금 우리가 Python으로 수식을 쓰듯, 당시에는 FORTRAN이 과학자들의 최고의 도구

프로그래밍 언어의 역사

버전	연도	특징
FORTTRAN I	1957	최초 버전, 수식 컴파일
FORTTRAN II	1958	서브루틴 추가
FORTTRAN IV	1962	표준화 시도
FORTTRAN 66	1966	ANSI 표준 채택
FORTTRAN 77	1977	IF문, 문자열 처리 도입
Fortran 90	1991	구조화, 모듈, 동적 메모리
Fortran 2003	2004	객체지향 요소 도입
Fortran 2008~2018	현대화 계속 중	

프로그래밍 언어의 역사

■ FORTRAN (1954)

✓ 현재

- 과학 시뮬레이션 및 수치 해석 (Numerical Simulation)
- 항공우주 및 국방 시뮬레이션
- 공학 계산 (Engineering Computing)
- 생물정보학 / 계산생물학
- 슈퍼컴퓨터 기반 병렬 계산
- NASA, ADD, 기상청, MIT, ..

```
PROGRAM HELLO  
PRINT *, 'HELLO, WORLD'  
END
```

프로그래밍 언어의 역사

■ FORTRAN (1954) – 현재 사용

이유	설명
막대한 레거시 코드 존재	기상청, 원자력연구소, 항공우주 연구소 등에서 수십 년 된 코드들이 Fortran으로 작성
고속 수치 계산	컴파일 최적화로 Python보다 훨씬 빠름
특화된 라이브러리	수치해석 전용 Fortran 라이브러리는 여전히 세계 최고 성능
안정성과 신뢰성	검증된 과학적 코드 자산이 많음 → 다시 수정 필요 없음

프로그래밍 언어의 역사

■ FORTRAN (1954) – 교육 전파 어려움

경로	설명
산업체 내부 교육	국책기관, 연구소, 엔지니어링 기업에서 자체 교육 진행 (예: 기상청, 항우연 등)
석·박사 과정 연구실 교육	과학 계산을 다루는 실험실에서 Fortran 기반 수치코드 학습
과학 컴퓨팅 전공 커리큘럼	일부 고급 수치해석 과목에서는 Fortran 실습 포함
레거시 코드 리팩토링 학습	변환 / 연동을 통해 접근

프로그래밍 언어의 역사

■ LISP (1958)

✓ 현재

■ 개발자: 존 매카시 (John McCarthy) - MIT

■ 특징:

- 인공지능(AI), 함수형 프로그래밍, 컴퓨터 과학 이론 전반에 지대한 영향을 준 핵심 언어
- 괄호 기반의 독특한 문법
(print "Hello, World")

1956년에 다트머스 학회에서 처음으로
인공지능(Artificial Intelligence)이라는
용어를 창안

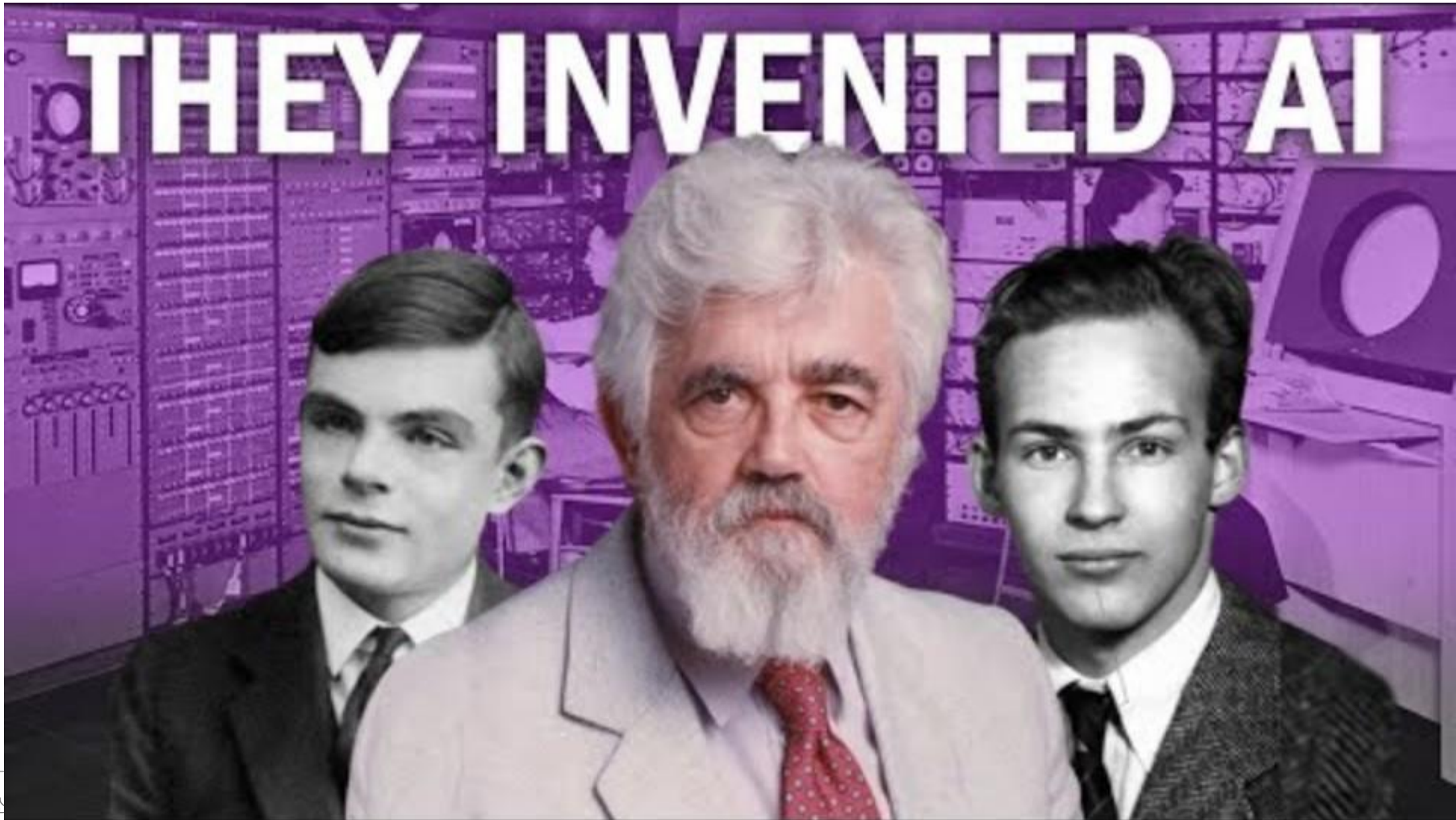
존 매카시
John McCarthy



존 매카시(2006)

출생	1927년 9월 4일 미국 U.S.
사망	2011년 10월 24일(84세) 미국 스탠퍼드
거주지	미국
국적	미국인
출신 학교	벨몬트 고등학교 (졸업, 1944) 캘리포니아 공과대학교 (학사, 1948) 프린스턴 대학교(박사, 1951)
주요 업적	인공지능, 리스프, 한계 지정

AI 초창기(1960~1980년대)에 전문가 시스템, 추론 엔진 개발에 LISP이 표준처럼 사용됨



프로그래밍 언어의 역사

■ ALGOL 60 (1960)

✓ 배경

- **FORTRAN은 상용화를 성공했지만, 구조적으로 복잡하고 표준이 불명확**
- 컴퓨터 과학자들은 수학적 알고리즘을 명확하고 추상적으로 표현할 수 있는 언어를 원함
- 유럽과 미국의 학자들이 협력해 프로그래밍의 '공통 표준 언어'를 만들자고 제안
- 특징:
 - 구조적 프로그래밍 개념 정립
 - 블록 구조와 중괄호 개념 도입
 - **C, Pascal 등에 직접적인 영향**

프로그래밍 언어의 역사

■ ALGOL 60 (1960)

✓ ALGOL 60이 끼친 영향

- C : 블록 구조, 제어문 구조
- Java/C++/C# : 함수 구조, 구조적 사고 방식
- Python : 절차적 흐름, 함수 정의, 지역 변수 등
- JavaScript : if, for, while, 함수 구조는 ALGOL 계열과 유사함

프로그래밍 언어의 역사

■ ALGOL 60 (1960)

✓ 의의

- 이론적 기초 제공 : 컴파일러, 문법 해석, 알고리즘 표현 등 핵심 개념 정립
- 현대 언어의 조상 : C 계열 언어들의 '문법 조상'
- 국제 표준화 첫 사례 : ACM (Association for Computing Machinery), 유럽 학회들이 공동 개발한 최초의 국제적 언어

✓ ??

- 상업적 지원 부족 (FORTRAN은 IBM의 강력한 후원)
- 실무용보다는 이론적 모델로 남음
- 컴파일러 구현이 어려움 (그 당시 기준으로 매우 진보적)

프로그래밍 언어의 역사

- ALGOL 60 (1960)

```
begin
  integer a, b, c;
  a := 3;
  b := 4;
  c := a + b;
  print(c)
end
```

프로그래밍 언어의 역사

■ COBOL (1959)

✓ 배경

- 당시 기업/정부 기관은 회계, 급여, 재무 계산을 컴퓨터로 처리하고자 함
- 하지만 대부분의 언어는 과학 계산(FORTRAN) 중심이라 **비즈니스 언어가 부재**
- 국방부가 IBM, Honeywell, RCA 등과 함께 공통된 비즈니스 언어를 만들기로 함

✓ 특징

- 영어 문장에 가까운 문법

DISPLAY "HELLO, WORLD".
ADD A TO B GIVING C.

프로그래밍 언어의 역사

■ COBOL (1959)

✓ 특징

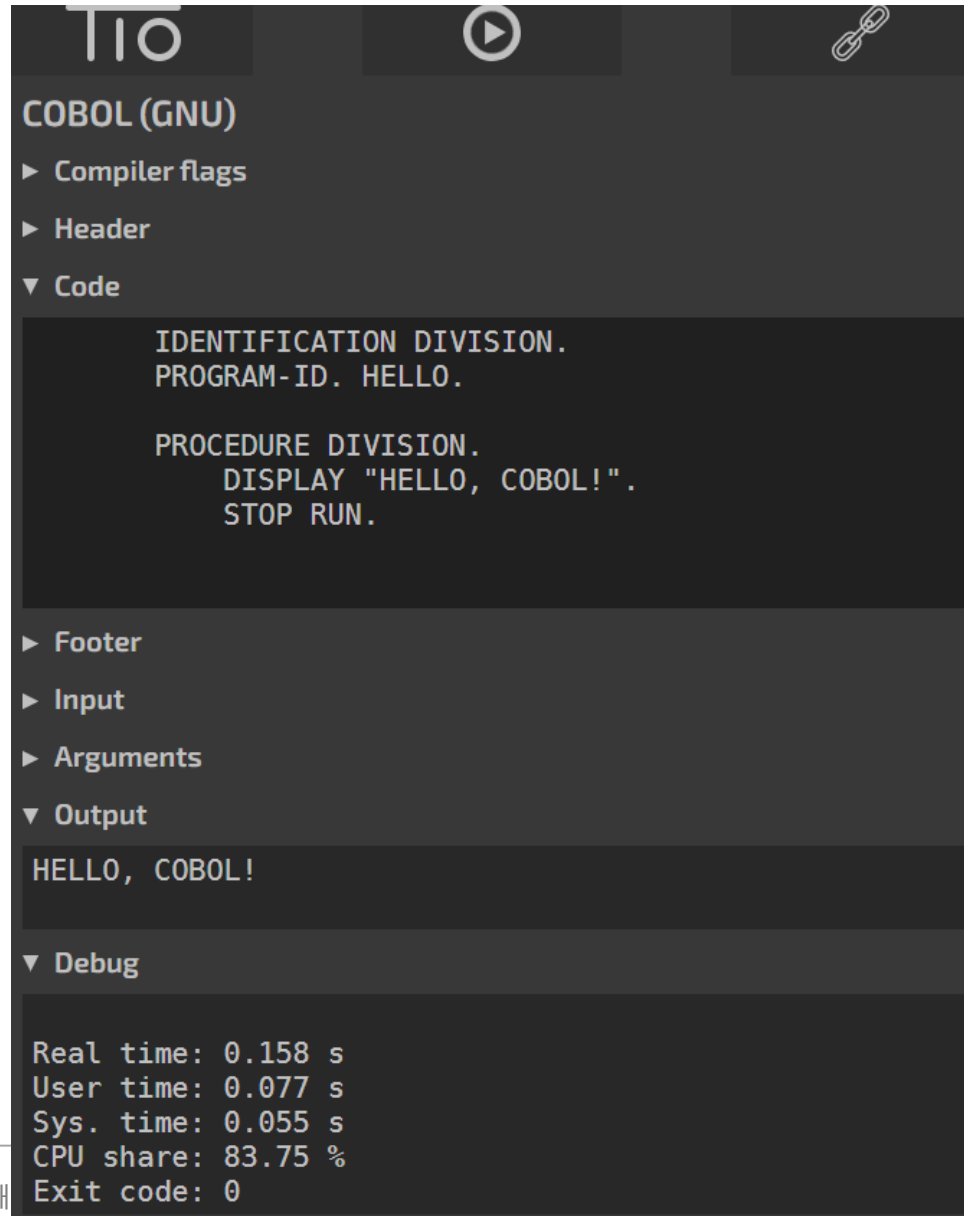
■ 비즈니스 처리에 최적화

- 대량의 파일 처리, 보고서 출력, 숫자 포맷 제어에 강력한 기능 제공
- 숫자 자리수 포맷(예: 999,999.99), 화폐, 날짜 등 명시적 제어 가능

✓ 활용 분야

- 은행
- 보험
- 급여

■ 회계, 재무



```
COBOL (GNU)
▶ Compiler flags
▶ Header
▼ Code
IDENTIFICATION DIVISION.
PROGRAM-ID. HELLO.

PROCEDURE DIVISION.
    DISPLAY "HELLO, COBOL!".
    STOP RUN.
▶ Footer
▶ Input
▶ Arguments
▼ Output
HELLO, COBOL!
▼ Debug
Real time: 0.158 s
User time: 0.077 s
Sys. time: 0.055 s
CPU share: 83.75 %
Exit code: 0
```

IDENTIFICATION DIVISION. // 프로그램 설명
PROGRAM-ID. HELLO. // 기본정보 id, 이름

PROCEDURE DIVISION. //실제 실행 부분
DISPLAY " HELLO, COBOL! ". // 화면 출력
STOP RUN. // 종료

프로그래밍 언어의 역사

■ PL/I (1964)

- ✓ 개발 주체 : IBM
- ✓ Full name: Programming Language One (PL/I = FORTRAN + COBOL + 새로운 기능)
- ✓ 목적 : 과학 계산용 + 비즈니스 처리용 기능을 통합한 범용 언어
- ✓ 특징
 - FORTRAN(수학/과학) + COBOL(비즈니스)의 장점 결합
 - 구조적 프로그래밍 지원, 예외 처리 지원
 - 포인터, 동적 메모리, 병행 처리 가능 (C언어 보다 먼저 등장)
 - 의의: 멀티태스킹과 범용성을 최초로 시도한 고급 언어

프로그래밍 언어의 역사

■ PL/I (1964)

✓ 실패

- 너무 많은 기능 : 복잡하고 무거운 언어로 비판 받음 (“모든 걸 다 하려다 과해짐”)
- IBM 전용 환경 : PL/I는 주로 IBM 메인프레임에서만 사용 가능 → 이식성 낮음
- 컴파일러 무겁고 느림 : 초기 컴파일러의 속도와 리소스 요구량이 너무 큼
- 무료 구현 부족 : 오픈소스 진영에서는 외면됨

프로그래밍 언어의 역사

- Simula = Simulation Language (1964)
 - ✓ 개발 주체 : 노르웨이 오슬로의 Nygaard & Dahl
 - ✓ 배경 : 과학/산업계에서는 은행, 병원, 공장 같은 시스템을 시뮬레이션하려는 수요 급증
 - ✓ 특징
 - 최초의 객체지향 언어 개념 제시
 - 클래스, 객체, 상속, 메서드, ..
 - 시뮬레이션용 언어에서 시작 → 객체지향 개념의 근간 제공
 - 오늘날 OOP 언어(C++, Java, Python 등)에 직접적인 철학적 기반을 제공

프로그래밍 언어의 역사

■ BASIC (1964)

- ✓ 개발자 : 존 케미니(John Kemeny), 토머스 커츠(Thomas Kurtz)
- ✓ 개발 장소 : 미국 다트머스 대학교 (Dartmouth College)
- ✓ 배경
 - 1960년대 당시 컴퓨터는 너무 고가이고 복잡해서 과학자, 수학자, 엔지니어들만 다룰 수 있는 도구
 - 복잡한 FORTRAN, ALGOL 등을 대체할 수 있는 **간단하고 친숙한 언어로 BASIC을 설계**
 - **비전공자들도 쉽게 컴퓨터 프로그래밍을 할 수 있도록 하기 위함**

프로그래밍 언어의 역사

■ BASIC (1964)

✓ 특징

- 초보자 친화적인 문법 : 문법이 자연어(영어)와 유사해서 배우기 쉽다
- 인터프리터 방식 : 코드를 한 줄씩 입력하고 바로 결과 확인 가능
- 줄 번호 기반 실행 : 각 줄에 줄 번호를 붙여 프로그램 흐름 제어
- 명확하고 제한된 키워드 : PRINT, INPUT, GOTO, IF ... THEN, FOR ... NEXT, LET, END
- 교육용 컴퓨터, 초기 게임 개발 등에서 사용
- 한계
 - 대규모 소프트웨어 개발에 부적합
 - 성능, 유지보수 어려움 / 앱 개발에서 미사용

```
10 PRINT "HELLO, BASIC!"  
20 END
```

Applesoft BASIC in Javascript

By Joshua Bell | [Source](#) | [README](#) — Applesoft BASIC Quick Reference

Related projects: [Logo in Javascript](#) | [Streaming video to an Apple II - vniIc](#)

```
HELLO, BASIC!
```

Enter code:

▶ Run

■ Stop

Select a sample...

```
10 PRINT "HELLO, BASIC!"  
20 END
```

Save

Load

Share

Show output

프로그래밍 언어의 역사

■ Pascal (1970)

- ✓ 개발자 : 니클라우스 에밀 비르트
- ✓ 개발 주체 : ETH Zurich (스위스 연방 공과대학)
- ✓ 배경
 - 1960년대 말, 프로그래밍 언어들이 점점 복잡하고 비체계적 시기
 - 비르트 교수는 명확한 구조와 타입 안정성을 갖춘 교육용 언어 필요성
 - ALGOL 언어의 철학을 계승하면서 단순화한 Pascal을 개발

니클라우스 에밀 비르트

독일어: Niklaus Emil Wirth



출생	1934년 2월 15일 스위스 빈터투어
사망	2024년 1월 1일(89세)
출신 학교	취리히 연방 공과대학교(B.S., 1959) 라발 대학교(M.S., 1960) 캘리포니아 대학교 버클리(Ph.D., 1963)

프로그래밍 언어의 역사

■ Pascal (1970)

✓ 특징

- 구조적 프로그래밍 강조
 - BEGIN ... END, IF ... THEN ... ELSE, FOR ... DO 등의 명확한 블록 구조
- 강한 타입 검사 (변수 선언 시 자료형을 반드시 명시)
- 교육용 및 학습용으로 널리 보급
 - 명확한 문법 구조 + 컴파일 기반 → 컴퓨터 구조와 언어 개념 학습에 이상적
- 많은 교육 기관에서 사용되며, Delphi의 기반

프로그래밍 언어의 역사

https://rextester.com/l/pascal_online_compiler

compile pascal online

Language: Pascal Layout: Vertical

```
1 program HelloWorld;  
2 begin  
3     writeln('Hello, Pascal!');  
4 end.  
5  
6
```

Run it (F8) Save it ☒ Show compiler warnings [+] Show input

Compilation time: 0.11 sec, absolute running time: 0.15 sec, cpu time: 0.01 sec, memory: 0.01 sec

Hello, Pascal!

```
program HelloWorld;  
begin  
    writeln('Hello, Pascal!');  
end.
```


프로그래밍 언어의 역사

■ SmallTalk (1971) – 최초의 GUI 언어

- ✓ 개발자 : Alan Kay, Dan Ingalls, Adele Goldberg
- ✓ 개발 주체 : , 미국 제록스(XEROX)사의 팰러앨토 연구 센터(PARC)에서 개발
- ✓ 목적 : 어린이도 프로그래밍을 할 수 있는 언어
- ✓ 특징
 - 순수 객체지향 언어 (C++, Java와 같은 객체 지향 프로그램의 시초)
 - 모든 것이 객체, 메시지 기반 통신
 - 오늘날 객체지향 프로그래밍(OOP), GUI 환경, 드래그 앤 드롭 UI, IDE(오늘날 개발 도구들의 뿌리)

프로그래밍 언어의 역사

■ C (1971)

✓ 개발자 : 데니스 리치 (Dennis Ritchie)

✓ 개발 주체 : 미국 Bell Labs (AT&T)

✓ 배경

- 당시 운영체제 개발에 사용되던 언어는 어셈블리어와 B(C 언어의 직계 조상)언어
- 어셈블리어는 이식성 부족 + 생산성 저하, B언어는 기능 미흡
- 데니스 리치는 B 언어를 개선하여 C 언어를 고안

B 언어 한계

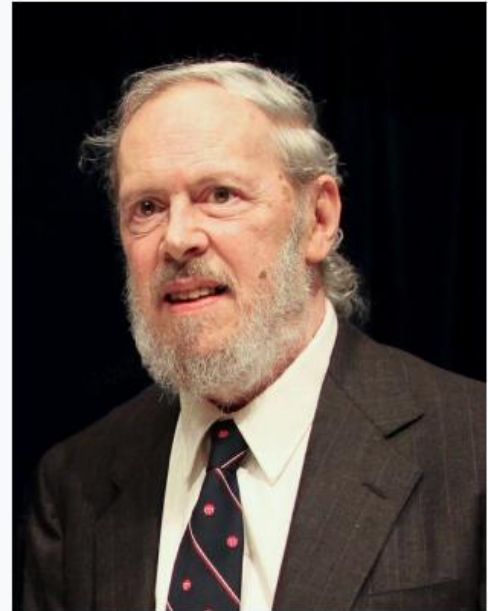
(B)데이터 타입 부족 : int, float, char (C)

(B)컴파일 성능 낮음 : 고속 컴파일러 개발 (C)

(B)비효율적 구조체 : 구조체 및 포인터 연산 개선 (C)

(B)이식성 제한 : 다양한 플랫폼에서 동작 개선 (C)

데니스 리치
Dennis Ritchie



데니스 리치(2011년)

출생	1941년 9월 9일 미국 뉴욕 브롱스빌
사망	2011년 10월 12일(70세) 미국 뉴저지 버클리 헤이츠
국적	미국
출신 학교	하버드 대학교 (Ph.D., 1968)

프로그래밍 언어의 역사

■ C (1971)

✓ 특징

- 고수준과 저수준의 융합
- 단순하고 효율적인 문법
 - 제어문: if, else, while, for, switch, break, continue
- 빠른 실행 속도
 - C 컴파일러는 효율적으로 최적화되어 기계어에 근접한 속도 제공
- 운영체제/시스템 프로그래밍 적합
 - UNIX, Linux 커널, 윈도우 커널 일부도 C로 작성
 - 드라이버, 펌웨어, 마이크로컨트롤러 프로그래밍에도 필수

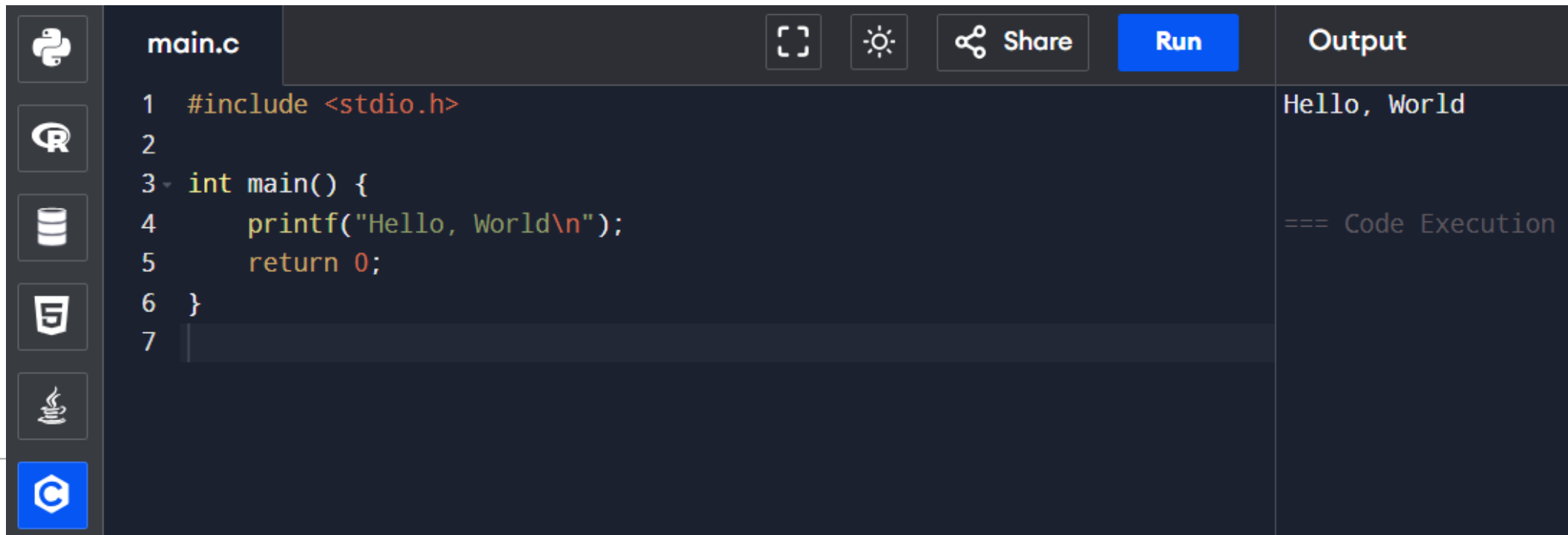
프로그래밍 언어의 역사

■ C (1971)

<https://www.programiz.com/c-programming/online-compiler/>

```
#include <stdio.h>
```

```
int main() {  
    printf("Hello, World\n");  
    return 0;  
}
```



The screenshot shows an online C compiler interface. On the left, there is a sidebar with icons for various programming languages: Python, R, SQL, JavaScript, Java, and C. The C icon is selected. The main area displays a code editor with the following C code:

```
main.c  
1  #include <stdio.h>  
2  
3  int main() {  
4      printf("Hello, World\n");  
5      return 0;  
6  }  
7
```

Below the code editor, there are buttons for "Run", "Share", and "Output". The "Run" button is highlighted in blue. To the right of the code editor, the "Output" section shows the result of the execution:

```
Hello, World  
  
=== Code Execution
```

프로그래밍 언어의 역사

■ C (1971)

✓ 영향

- C ++ : 객체지향 기능 추가 (C 기반)
- Java : 기본 문법 구조 (조건문, 반복문 등)
- C# : C + .NET 스타일
- Python : 문법 유사

비운의 과학자 데니스 리치



프로그래밍 언어의 역사

■ Ada (1980)

- ✓ 개발자 : 미 국방부 (DoD), 나토(NATO)
- ✓ 이름 유래 : 세계 최초의 프로그래머 "Ada Lovelace"에서 유래
- ✓ 목적 : 군사·항공 등 고신뢰 시스템 개발을 위한 안전한 고급 언어 필요성 대응
- ✓ 문제점
 - 미 국방부에서 많은 프로그래밍 언어를 혼용
 - 유지보수 불가능, 재사용성 낮음, 버그 잦음 → 군사/항공용 소프트웨어 품질이 생명과 직결되는 만큼, 단일 고급언어 필요
- ✓ 특징
 - 변수 타입 검사, 안정성, 모듈성, 병렬성, 예외 처리 강조

프로그래밍 언어의 역사

■ C++ (1983)

- ✓ **개발자** : 비야네 스트롭스트롭 (Bjarne Stroustrup)
- ✓ **개발 주체** : AT&T 벨 연구소 (Bell Labs)
- ✓ **배경**
 - C 언어는 빠르고 강력하지만, 복잡한 소프트웨어 구조화에는 한계가 있었음
 - 객체지향 언어의 철학을 C에 도입하여 성능과 추상화 구조를 모두 갖춘 언어를 만들자는 목표

비야네 스트롭스트롭



본명	Bjarne Stroustrup
출생	1950년 12월 30일(74세) 덴마크 오르후스
거주지	미국 뉴욕주 뉴욕 시
성별	남성
국적	덴마크

프로그래밍 언어의 역사

■ C++ (1983)

✓ 특징

- 객체지향 프로그래밍(OOP) : 캡슐화, 상속,...
- C 언어 완전 호환 : 기존 C 프로그램을 그대로 컴파일할 수 있음
- 함수 오버로딩 & 연산자 오버로딩

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

```
#include <iostream>
```

```
int main() {
    std::cout << "Hello" << std::endl;
    return 0;
}
```

프로그래밍 언어의 역사

■ Perl (1987)

- ✓ 개발자 : 래리 월 (Larry Wall)
- ✓ 목적 : 시스템 관리자가 로그 파일, 텍스트 등을 쉽게 가공할 수 있도록 설계
- ✓ 배경
 - 래리 월은 시스템 관리자였고, 다양한 로그 파일을 분석하거나 레포트를 자동화하는 작업을 수행
 - 당시 사용하던 awk, sed, sh만으로는 복잡한 텍스트 처리가 어려움

→ 그것들을 모두 합친 '만능 스크립트 언어'를 만들자 라는 생각에서 Perl이 탄생

```
my $name = "Alice";  
my $age = 25;  
print "$name is $age years old\n";
```

compile perl online

Language: Layout:

```
1 my $name = "Alice";  
2 my $age = 25;  
3 print "$name is $age years old\n";  
4
```

[+] Show input

Absolute running time: 0.16 sec, cpu time: 0.01 sec, memory peak: 6 Mb, absolute service time: 0,28 sec

Alice is 25 years old

https://rextester.com/l/perl_online_compiler

```
my $name = "Alice";  
my $age = 25;  
print "$name is $age years old\n";
```

프로그래밍 언어의 역사

■ HTML (1990)

- ✓ Full name: HyperText Markup Language
- ✓ 개발 주체 : 팀 버너스리 (Tim Berners-Lee) - 유럽입자물리연구소(CERN, 스위스 제네바 소재)
- ✓ 배경 :
 - 연구자들은 수많은 문서를 이메일, FTP, 프린트물 등으로 비효율적으로 공유
 - 컴퓨터 종류나 네트워크 환경에 상관없이, 모든 문서를 하이퍼링크로 연결하고, 웹 브라우저 하나만 있으면 전 세계의 정보에 접근할 수 있다면 어떨까?
- ✓ 특징
 - 웹페이지의 구조와 내용을 표현하는 마크업 언어
 - 프로그래밍 언어는 아니지만 웹 개발의 기본, 태그 기반 (<html>, <body>, <p>, <a> 등)

프로그래밍 언어의 역사

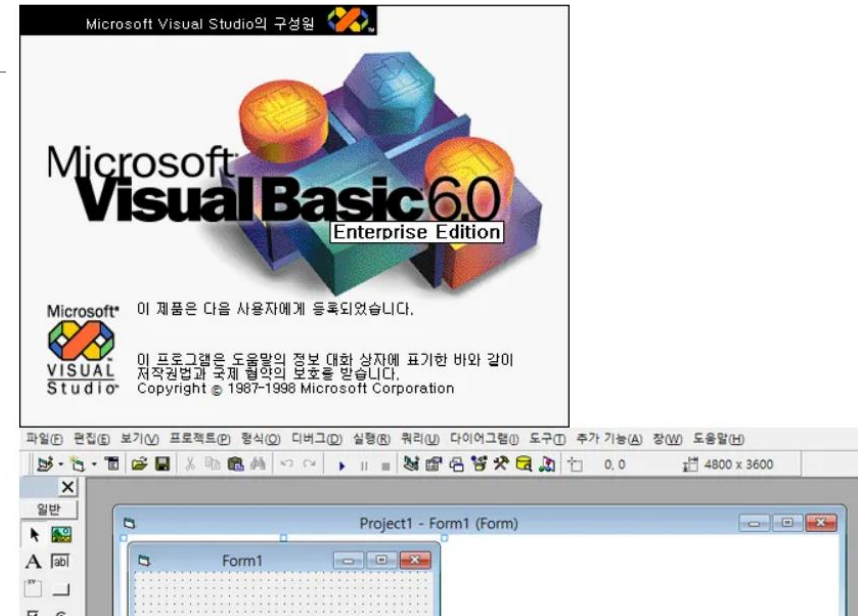
■ Visual Basic (1991)

✓ 개발자 : MS

✓ 특징

- 윈도우 기반 GUI 앱 제작에 최적화
- 드래그 앤 드롭 방식 + 코드 연결로 개발이 쉬움
- 비개발자도 쉽게 접근 가능
- 의의: Rapid Application Development(RAD)의 대표 언어
- 기업용 MS 데스크탑 어플리케이션 개발에 활용 ()

2.3. Visual Basic 6.0



프로그래밍 언어의 역사

■ Python (1991)

- ✓ 개발자 : 귀도 반 로섬 (Guido van Rossum)
- ✓ 목적 : 누구나 쉽게 배워서 무엇이든 만들 수 있도록 하자 (철학)
- ✓ 특징
 - 간결하고 읽기 쉬운 문법
 - 세미콜론(;) 없음
 - 들여쓰기로 블록 구분 (가독성 중심)
 - 복잡한 문법 최소화

귀도 반 로섬



2024년 PyCon에서 반 로섬

출생명	Guido van Rossum
출생	1956년 1월 31일(69세) ^[1] 네덜란드 헤이그 ^[2]
성별	남성
직업	컴퓨터 프로그래머, 저자
배우자	킴 냅(결혼: 2000년)
자녀	1 ^[3]

공학 경력

고용주 [마이크로소프트](#)

프로그래밍 언어의 역사

■ Python (1991)

✓ 특징

- 인터프리터 언어
 - 컴파일 없이 한 줄씩 실행
 - 빠른 테스트 및 디버깅에 유리

✓ 활용 분야

- 웹 개발
- AI/머신러닝
- 교육용 프로그램

```
x = 10      # int
x = "hello" # str (동적으로 바뀜)
```

■ 자동화 스크립트...



프로그래밍 언어의 역사

■ JAVA (1994)

✓ 개발사 : Sun Microsystems (→ 현재는 Oracle 소유)

✓ 개발자 : 제임스 고슬링(James Gosling)

✓ 배경

■ 네트워크 시대에 맞춰 안정적이고 이식성 있는 객체지향 언어가 필요했음

■ 가전제품/네트워크 장치에서도 작동할 수 있는 범용 언어를 목표로 개발

James Arthur Gosling

제임스 아서 고슬링



국적	 캐나다
출생	1955년 5월 19일 (70세) 캐나다 자치령 앨버타 주 캘거리
학력	캘거리 대학교 (컴퓨터과학 / 학사) 카네기 멜런 대학교 (컴퓨터과학 / 석사, 박사)
직업	소프트웨어 개발자, 컴퓨터과학자
소속	아마존 웹 서비스, 오라클, 구글, 썬 마이크로시스템즈
수상	Officer of the Order of Canada

■ JAVA (1994)

✓ 특징

- 플랫폼 독립성 (JVM 기반)
- 완전한 객체지향 언어
- 강력한 표준 라이브러리

- 파일 입출력, 네트워크, 데이터베이스, 쓰레드, 암호화 등 기능 내장

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

활용분야

- 웹 개발
- 안드로이드 앱
- 데스크탑 앱
- IOT 프로그램
- 게임 등

프로그래밍 언어의 역사

■ Delphi (1995)

- ✓ 개발자 : 앤더스 하일스버그 (Anders Hejlsberg) C#, .NET Framework 설계 총괄
- ✓ 용도 : Windows 데스크탑 GUI 프로그램, DB 애플리케이션 개발
- ✓ 특징
 - Object Pascal 기반
 - 고속 컴파일러
 - 데이터베이스 연동 최적화
 - RAD (Rapid Application Development) 도구 내장 — 빠른 GUI 설계 가능

안데르스 하일스버그



2008년의 하일스베르크

태어나 1960년 12월 2일 (64세)
다 코펜하겐, 덴마크
국적 덴마크 말
교육 덴마크 공과대학교 [1]
직업 프로그래머
(들) 시스템 아키텍트
고용주 마이크로소프트 [1]

프로그래밍 언어의 역사

■ Delphi (1995)

✓ 특징

- Windows에 최적화된 앱 제작 : 설치형 실행파일(.exe) 생성, UI 자동 구성
- 간편한 DB 연동 : TTable, TQuery, TDataSource 등 DB 컴포넌트만 연결해도 동작
- 빠른 배포 : 복잡한 설치 과정 없이 바로 실행 가능한 앱 생성
- 비개발자도 접근 가능 : 디자이너 중심 UI + 코드 자동 생성
- **1990년대 후반, Visual Basic과 함께 데스크탑 프로그래밍의 양대 산맥**

■ JavaScript (1995)

✓ 개발자 : 브렌든 아이크 (Brendan Eich, Netscape)

✓ 목적 : 웹 브라우저에서 동작하는 동적 프로그래밍 언어

✓ 특징

- 브라우저 내장 언어 : HTML, CSS와 함께 웹 3대 언어
- 인터프리터 기반
- 객체 기반 언어
- 이벤트 중심 프로그래밍 : 마우스 클릭, 키보드 입력 등 처리
- 발전

- Node.js(서버 프로그래밍(백엔드 개발 가능)),
- React,..(프론트엔드 프레임워크)로 웹 전체를 장악

```
let score = 85;
if (score >= 90) {
    console.log("A");
} else {
    console.log("B or lower");
}
```

브랜든 아이크
Brendan Eich



출생	1961년 7월 4일 (64세) 미국 펜실베이니아 주 피츠버그
국적	미국
학력	산타클라라 대학교 (수학/컴퓨터과학 학사) 일리노이 대학교/어배너-섐페인 캠퍼스 (컴퓨터과학 / 석사)
직업	프로그래머

프로그래밍 언어의 역사

■ PHP (1995) <https://www.programiz.com/php/online-compiler/>

- ✓ 개발자: 라스무스 러도프(Rasmus Lerdorf)
- ✓ 목적 : 서버 사이드 웹 프로그래밍 언어 (HTML에 삽입 가능)
- ✓ 특징
 - 서버 사이드 실행
 - HTML과 결합 가능
 - 간단한 문법 : C, Perl 문법 기반 → 배우기 쉬움
 - 웹 개발 최적화 등

```
<?php
    $score = 85;
    if ($score >= 90) {
        echo "A";
    } else {
        echo "B or lower";
    }
?>
```

라스무스 러도프
Rasmus Lerdorf

로마자 표기

Rasmus Lerdorf

출생

1968년 11월 22일
그린란드

성별

남성

국적

덴마크계 캐나다인

경력

PHP 스크립트 언어 창시

직업

프로그래머

급하였

1995년 12월 1일



2014년의 라스무스 러도프

프로그래밍 언어의 역사

- XML(eXtensible Markup Language) (1996)
 - ✓ Full name: eXtensible Markup Language
 - ✓ 개발 주체: **W3C (World Wide Web Consortium)**
 - ✓ 특징
 - 데이터 저장 및 교환을 위한 마크업 언어
 - 사용자 정의 태그 사용 가능 → 다양한 데이터 표현
 - 구조적, 계층적 데이터 표현
 - UI 레이아웃 파일 (res/layout/*.xml), 리소스 정의 등

프로그래밍 언어의 역사

■ C# (2000)

- ✓ 개발 주체: Microsoft (Anders Hejlsberg 주도)
- ✓ 개발자 : **Anders Hejlsberg** (델파이, 터보 파스칼 개발자)
- ✓ 목적: **Java** 대항마로 **.NET** 기반 통합 언어 개발
- ✓ 특징
 - 객체지향 + 함수형 + 이벤트 기반 + 비동기 프로그래밍 지원
 - Windows 앱, 게임(Unity), 웹, 클라우드 등 폭넓게 사용
- ✓ 활용
 - Windows 데스크탑 앱, 클라우드 개발
 - 모바일 앱, 사내 자동화 툴

프로그래밍 언어의 역사

■ VB. NET (2002) - Visual Basic 6.0의 후속 언어

✓ 개발 주체: Microsoft (Visual Basic의 .NET 버전)

✓ 특징

- Visual Basic 문법 + .NET Framework 기반
- 객체지향 완전 지원
- Windows 앱 개발 최적화
- 웹, 네트워크, 클라우드 개발

프로그래밍 언어의 시대별 발전 흐름

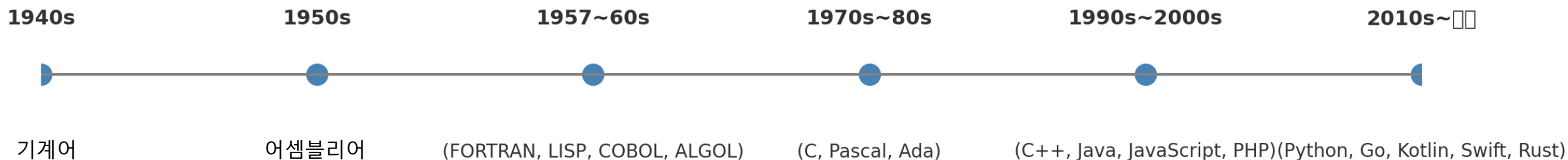
- 1세대 (1940s): 기계어 (0, 1)
- 2세대 (1950s): 어셈블리어 – 기계어를 기호화
- 3세대 (1957~1960s): 고급 언어 (FORTRAN, COBOL, LISP, ALGOL)
- 4세대 (1970s~1980s): 구조적/시스템 언어 (C, Pascal, ..)
- 5세대 (1990s~2000s): 객체지향 & 웹 언어 (C++, Java, JavaScript, PHP)
- 현대 (2010s~현재): 데이터/AI/모바일 언어 (Python, Kotlin, Swift, Rust)

프로그래밍 언어 패러다임 비교

- 절차적 프로그래밍: 순차적으로 명령을 실행 (예: C, Pascal)
- 객체지향 프로그래밍(OOP): 객체 단위로 데이터+기능 묶음 (예: Java, C++)
- 함수형 프로그래밍: 수학적 함수처럼 상태 변화 최소화 (예: LISP, Python, ..)

프로그래밍 언어 시대별 발전 흐름 타임라인

프로그래밍 언어 시대별 발전 흐름 타임라인



퀴즈 -1

1. 세계 최초의 고급 프로그래밍 언어로, 과학 계산에 특화된 언어는?

- ① FORTRAN
- ② COBOL
- ③ BASIC
- ④ LISP

2. Plankalkül 언어를 개발한 사람은 누구인가?

- ① 콘라트 츠제 (Konrad Zuse)
- ② 존 매카시 (John McCarthy)
- ③ 존 백커스 (John Backus)
- ④ 데니스 리치 (Dennis Ritchie)

퀴즈 -3

3. “COmmon Business-Oriented Language”라는 이름을 가진 언어는?

- ① COBOL
- ② FORTRAN
- ③ PL/I
- ④ Pascal

퀴즈 -4

4. 인공지능(AI) 연구를 위해 개발된 언어로, 괄호 기반 문법과 리스트 자료구조가 특징인 언어는?

- ① LISP
- ② Logo
- ③ APL
- ④ Scheme

5. 시스템 프로그래밍과 운영체제(UNIX) 개발에 사용된 언어는?

- ① Python
- ② C
- ③ Pascal
- ④ PHP

퀴즈 -6

6. “Write Once, Run Anywhere”라는 철학을 가진 언어로, JVM 위에서 실행되는 언어는?

- ① JavaScript
- ② Java
- ③ Kotlin
- ④ C#

퀴즈 -7

7. 웹 브라우저에서 동작하며, 프론트엔드 프로그래밍의 핵심 언어는?

- ① PHP
- ② JavaScript
- ③ Ruby
- ④ Visual Basic

퀴즈 -8

8. 웹 서버 사이드 프로그래밍 언어로, HTML에 직접 삽입해 실행할 수 있는 언어는?

- ① Perl
- ② PHP
- ③ Python
- ④ Ruby

9. 최근 AI와 데이터 과학 분야에서 가장 널리 사용되는 언어는?

- ① Java
- ② Python
- ③ C++
- ④ Go

Thank you for Listening

새로운 세상과 변화에 도전하는 두원인이 되기를 바랍니다.